2026-10-31 17:30:19 You

now for each topics pick two most commonly done task by ansible developers and comeup with a storytelling like short para of how i worked on that may be at last add technical details also simply and at verylast give me a memory connecting things to connect all points and story

ChatGPT gpt-6-mini

Let's build this as a **real production experience story**, not just a list of Ansible commands.

Imagine you're working on an application that serves lakhs of users across India. Your team manages multiple Linux servers on AWS EC2, divided into development, QA, UAT and production environments.

Your responsibility is to automate server administration, maintain consistent configurations, support deployments and reduce manual operational work.

For each area, I'll give you:

1. **Two common production tasks**
2. **How to explain your involvement in an interview**
3. **Technical details** – modules, commands and implementation.

The goal is to understand the story so you can answer naturally when the interviewer asks follow-up questions.

1. Linux Server Administration

Task 1: User and access management

Story:

"In our project, we regularly had requirements to provide server access to developers and support engineers. Instead of manually creating users on individual servers, we used Ansible to automate user creation, group assignment, SSH access and sudo permissions. This helped us maintain consistent access across multiple servers and made user onboarding and offboarding easier."

Technical details:

- Modules: `user`, `group`, `authorized_key`, `file`
- Create users and assign groups.
- Configure SSH public keys.

- Manage sudo permissions.
- Remove access when employees leave.

Task 2: Filesystem and directory management

Story:

"We also used Ansible to maintain standard directory structures across application servers. For example, application logs, deployment packages and application configurations were maintained in predefined locations. We automated directory creation, ownership and permissions so that applications could access the required files without permission-related issues."

Technical details:

- Modules: `file`, `mount`
- Manage `/opt/app`, `/var/log/app`, `/data`.
- Configure ownership using `owner` and `group`.
- Set permissions using `mode`.
- Manage filesystem mounts where required.

2. Package & Dependency Management

Task 1: Application dependency installation

Story:

"Whenever we provisioned a new application server, we needed to install the required runtime dependencies. We used Ansible to automate the installation of Java, Python and other application dependencies. We maintained the required versions through variables and reusable roles, ensuring that all servers had consistent runtime environments."

Technical details:

- Modules: `package`, `apt`, `dnf`, `yum`
- Install Java, Python, Git, curl, etc.
- Maintain specific package versions.
- Use variables for version management.

Task 2: OS patching and updates

Story:

"As part of regular server maintenance, we performed OS patching to address security vulnerabilities and keep systems updated. We used Ansible to automate package updates across server groups. For production, we followed a controlled rollout, updating a limited number of servers at a time, validating their health before proceeding with the remaining servers."

Technical details:

- Modules: `dnf`, `apt`, `package`
- Commands: `ansible-playbook`, `--limit`, `--check`
- Use rolling execution with `serial`.
- Validate services after patching.
- Maintain rollback or recovery plans.

3. Service Management

Task 1: Application service management

Story:

"We managed application services running on Linux servers using systemd. Whenever there was a configuration change or application deployment, we used Ansible to restart or reload the required services. We also ensured that critical application services were enabled at boot time, so they would automatically start after server reboots."

Technical details:

- Module: `systemd_service` (or `service`)
- Start, stop, restart and reload.
- Enable services on boot.
- Validate using `systemctl status`.

Task 2: Web server configuration

Story:

"We also automated the configuration of web servers like Nginx. We maintained standard configuration templates and used Ansible to deploy them across application environments. Whenever a configuration changed, Ansible updated the configuration, validated it and restarted or reloaded Nginx only when necessary."

Technical details:

- Modules: `template`, `copy`, `systemd_service`
- Nginx configuration in `/etc/nginx/`.
- Validate using `nginx -t`.
- Use handlers for conditional reloads.

4. Security & Compliance

Task 1: SSH hardening and firewall configuration

Story:

"Security was an important part of server administration. We used Ansible to maintain SSH configurations across Linux servers, such as restricting root login, controlling password-based authentication and managing authorized access. We also automated firewall rules to ensure that only required ports were accessible."

Technical details:

- Modules: `lineinfile`, `template`, `firewalld`, `ufw`
- SSH configuration: `/etc/ssh/sshd_config`
- Restrict root login.
- Configure allowed ports such as 22, 80 and 443 according to requirements.
- Validate SSH connectivity before applying changes broadly.

Task 2: Permission and security policy management

Story:

"We also maintained security-related configurations such as file permissions, sudo privileges and ownership. For example, application directories should be accessible only to the required application users rather than everyone on the server. We automated these settings through Ansible to maintain consistent security policies."

Technical details:

- Modules: `file`, `user`, `group`, `lineinfile`
- Apply least-privilege access.
- Manage sudoers configurations.
- Restrict sensitive configuration files.
- Use Ansible Vault for credentials and sensitive variables.

5. Application Deployment & Configuration

Task 1: Application deployment

Story:

"During application releases, we used Ansible to automate the deployment of application artifacts to Linux servers. Once the build was generated through Jenkins, the deployment process involved transferring the JAR or WAR file, placing it in the appropriate application directory, updating the required configurations and restarting the application service. We then validated whether the application was running successfully."

Technical details:

- Modules: `copy`, `get_url`, `file`, `systemd_service`
- Artifact source: Jenkins build output or artifact repository.
- Deployment location: `/opt/app/`.
- Manage application service restarts.
- Perform post-deployment health checks.

Task 2: Environment-specific configuration

Story:

"We maintained different configurations for development, QA and production environments. Instead of manually modifying configuration files on each server, we used Ansible variables and Jinja2 templates. This allowed us to maintain one reusable playbook while supplying different database endpoints, application ports and environment variables based on the target environment."

Technical details:

- Modules: `template`, `copy`
- Jinja2 templates: `.j2`
- Variables: `group_vars`, `host_vars`.
- Inventory groups: `dev`, `qa`, `prod`.
- Ansible Vault for sensitive values.

6. Monitoring & Logging

Task 1: Monitoring agent installation

Story:

"We automated monitoring agent installation across our Linux infrastructure. For example, whenever a new EC2 instance was provisioned, Ansible installed and configured the required monitoring agents, such as CloudWatch Agent or Node Exporter. We also ensured that the agents were running as system services so that infrastructure metrics could be collected continuously."

Technical details:

- Modules: `package`, `copy`, `template`, `systemd_service`
- Install CloudWatch Agent / Node Exporter.
- Configure CPU, memory and disk metrics.
- Configure agent startup on reboot.
- Validate agent status.

Task 2: Log forwarding configuration

Story:

"We also automated log collection configurations across application servers. We installed and configured log forwarding agents such as Fluent Bit or Filebeat to collect application and system logs and forward them to a centralized logging platform. This helped the support team investigate production issues without manually logging into every server."

Technical details:

- Modules: `package`, `template`, `systemd_service`
- Configure log source paths.
- Set up log forwarding destinations.
- Manage log rotation using `logrotate`.
- Validate agent connectivity and log delivery.

7. Infrastructure & Environment Management

Task 1: EC2 post-provisioning configuration

Story:

"In our AWS environment, EC2 instances were provisioned through the infrastructure provisioning process. After instance creation, we used Ansible for post-provisioning configuration. This included installing dependencies, creating application users, configuring directories, setting up monitoring agents and preparing the server for application deployment."

Technical details:

- AWS EC2 + Ansible.
- Dynamic inventory using AWS EC2 inventory plugin.
- Group servers using tags such as `Environment` and `Application`.
- Configure servers through reusable roles.
- AWS Systems Manager or SSH for connectivity, depending on the setup.

Task 2: Environment standardization

Story:

"We maintained separate development, QA and production environments. One of our important responsibilities was ensuring that the base server configurations remained consistent across these environments. We maintained common Ansible roles and used environment-specific variables to avoid configuration drift and reduce manual errors."

Technical details:

- Separate inventory files.
- `group_vars` and `host_vars`.
- Reusable roles.

- Environment-specific playbook execution.
- Configuration validation and drift correction.

8. Backup & Disaster Recovery

Task 1: Backup automation

Story:

"We automated routine backup-related activities using Ansible. For example, we configured scheduled backup scripts and cron jobs to back up important application configuration files and selected directories. We also maintained backup locations and permissions to ensure that backup operations could execute consistently."

Technical details:

- Modules: `cron`, `copy`, `synchronize`, `file`
- Schedule backups using cron.
- Store backups in designated directories or S3.
- Maintain retention policies.
- Validate backup completion.

Task 2: Server recovery and reconfiguration

Story:

"In case of server replacement or recovery, we used Ansible to recreate the required server configuration. Instead of manually repeating all installation and configuration steps, we could execute the existing playbooks to install dependencies, create users, configure services and restore application configurations. This reduced the time required to bring replacement servers into service."

Technical details:

- Re-execute infrastructure configuration playbooks.
- Restore required configurations and backups.
- Reinstall monitoring agents.
- Reconfigure application services.
- Validate application health after recovery.

9. Performance & Troubleshooting

Task 1: Disk-space management

Story:

"We used Ansible to automate routine disk-space checks across Linux servers. For example, when log files or temporary files started consuming significant disk space, we used automation to identify large directories, check disk utilization and perform approved cleanup activities. This helped prevent disk-related application failures."

Technical details:

- Modules: `command`, `shell`, `find`, `file`
- Commands: `df -h`, `du -sh`, `lsblk`.
- Identify large log and temporary files.
- Apply controlled cleanup policies.
- Avoid deleting active application logs or important data.

Task 2: Service health checks and recovery

Story:

"We also automated basic service health checks across application servers. Ansible was used to verify whether critical services were running and whether application endpoints were responding. If a service was found to be stopped, we could execute controlled recovery tasks, restart the service and validate its status."

Technical details:

- Modules: `service_facts`, `systemd_service`, `uri`, `wait_for`
- Check systemd service status.
- Validate application health endpoints.
- Restart failed services where appropriate.
- Capture output and report failures.

10. Automation & Orchestration

Task 1: Reusable roles and playbooks

Story:

"As the number of servers increased, maintaining individual automation scripts became difficult. So we organized our automation into reusable Ansible roles. For example, we maintained separate roles for Linux configuration, Java installation, Nginx configuration and monitoring agent setup. These roles could be reused across different environments, making automation easier to maintain."

Technical details:

Typical role structure:

```
</> Plain text

roles/
  linux_setup/
    tasks/main.yml
    handlers/main.yml
    templates/
    files/
    vars/main.yml

  java_install/
    tasks/main.yml

  monitoring/
    tasks/main.yml
```

Task 2: Jenkins and Ansible integration

Story:

"We also integrated Ansible with Jenkins to automate configuration and deployment activities. Whenever a release was approved, Jenkins triggered the deployment pipeline, generated the application artifact and invoked the Ansible playbook to deploy it to the target servers. This reduced manual deployment steps and improved consistency across releases."

Technical details:

- Jenkins pipeline.
- Git repository for Ansible playbooks.
- Jenkins credentials for secure access.
- Execute `ansible-playbook`.
- Pass environment variables and inventory.
- Validate deployment and report pipeline status.

Example:

```
</> Bash

ansible-playbook -i inventory/prod deploy.yml
```

---

11. Connect everything into one production story

This is the most important part. Instead of remembering 20 unrelated tasks, understand how they connect during a real production workflow.

Imagine your company launches a new application environment.

Production Ansible workflow

1. **EC2 server provisioning**AWS team provisions new Linux EC2 instances.
2. **Linux server preparation**Ansible creates users, groups, directories, permissions and SSH configurations.
3. **Dependency installation**Installs Java, Git, Python and other required packages.
4. **Security configuration**Applies SSH hardening, firewall rules and access policies.
5. **Application configuration**Deploys JAR/WAR files, updates environment configurations and configures Nginx.
6. **Service management**Starts application services and validates their status.
7. **Monitoring setup**Installs CloudWatch Agent, Node Exporter and log forwarding agents.
8. **Ongoing maintenance**Performs patching, backups, disk checks and troubleshooting.
9. **CI/CD integration**Jenkins triggers Ansible playbooks during automated deployments.

12. Memory trick: Remember Ansible through 5 stages

You don't need to memorize all 20 tasks individually. Connect them to five stages.



1. PREPARE

Users, groups, directories, packages, dependencies.

Question: How do I prepare a server?



2. SECURE

SSH, firewall, permissions, sudo, Vault.

Question: How do I secure a server?



3. DEPLOY

Application artifacts, configurations, Nginx, systemd.

Question: How do I make the application run?



4. MONITOR & MAINTAIN

Agents, logs, patching, backups, disk checks, recovery.

Question: How do I keep the application healthy?

5. AUTOMATE

Roles, inventories, playbooks, Jenkins integration, orchestration.

Question: How do I repeat everything consistently?

13. Technical commands you should recognize

Requirement	Ansible command
Test connectivity	`ansible all -m ping`
Execute playbook	`ansible-playbook site.yml`
Dry run	`ansible-playbook site.yml --check`
Check differences	`ansible-playbook site.yml --diff`
Specific server	`ansible-playbook site.yml --limit web01`
Specific tags	`ansible-playbook site.yml --tags nginx`
Ask for Vault password	`ansible-playbook site.yml --ask-vault-pass`
Syntax validation	`ansible-playbook site.yml --syntax-check`

14. Test your production understanding

Try answering these without looking at the explanations.

Production interview practice 1 / 6

A new EC2 instance is launched. How do you prepare it for application deployment?

Reveal answer Next question →

Restart practice

My advice for your client interview: Focus especially on application deployment, Linux administration, package management, service management, monitoring agents, reusable roles and Jenkins integration. These connect directly to day-to-day production operations.

One important distinction: Treat these as interview-ready example narratives. Only describe tasks as your actual hands-on experience if you have performed them; otherwise explain them as workflows you understand and can implement.

Your central Ansible story should always be:

"We provision the server, prepare it, secure it, configure the application, deploy it, monitor it and maintain it — and Ansible helps us automate these activities consistently across multiple

environments."